

Introduction to Support Vector Machines - Practical Notebook Walk Through

September 24, 2026

```
[1]: # Importing the libraries
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, \
    accuracy_score, f1_score, recall_score
import shap
import pyarrow
```

```
[3]: # Reading the dataset
```

```
df = pd.read_parquet('final_dataset.parquet')
```

```
# Replacing NaN values with 0
df = df.fillna(0)
```

```
[4]: df.head()
```

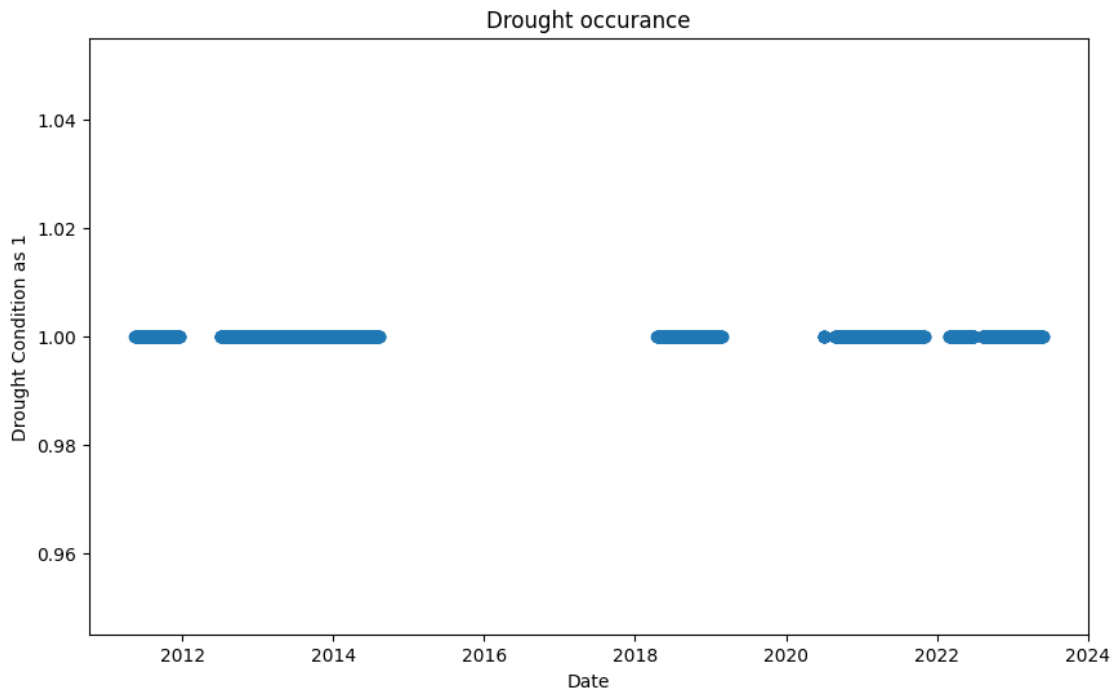
```
[4]:
```

	date	precip	max_temp	min_temp	gwl	daily_discharge_cfs	\
0	2010-02-14	0.0	36	13	20.52	136.0	
1	2010-02-15	0.0	41	7	20.51	129.0	
2	2010-02-16	0.0	43	8	20.51	130.0	
3	2010-02-17	0.0	51	10	20.51	132.0	
4	2010-02-18	0.0	51	16	20.50	135.0	

	sm_pca	st_pca	target
0	3.619512	0.190574	0
1	3.470118	0.209177	0
2	3.433721	0.197542	0
3	3.351107	0.135033	0
4	3.238121	0.107690	0

```
[5]: # Visualizing where target variable is 1 i.e there is drought
```

```
plt.figure(figsize=(10, 6))
plt.scatter(df[df['target'] == 1]['date'], df[df['target'] == 1]['target'])
plt.title('Drought occurance')
plt.xlabel('Date')
plt.ylabel('Drought Condition as 1')
plt.show()
```



```
[6]: # following are the features list
```

```
features = ['precip', 'max_temp', 'min_temp', 'gwl', 'daily_discharge_cfs', 'sm_pca', 'st_pca']
target = 'target'
```

```
[7]: # Checking for multicollinearity between features
```

```
corr = df[features].corr()
corr.style.background_gradient(cmap='coolwarm')
```

```
[7]: <pandas.io.formats.style.Styler at 0x7f82bb61bfa0>
```

We can see that there is correlation between max_temp and min_temp, but as per the domain knowledge, we know that these two features are important and we should keep them

```
[8]: # Displaying the data before splitting
```

```
display(df[features].head())
```

```
display(df[[target]].head())
```

	precip	max_temp	min_temp	gwl	daily_discharge_cfs	sm_pca	st_pca
0	0.0	36	13	20.52	136.0	3.619512	0.190574
1	0.0	41	7	20.51	129.0	3.470118	0.209177
2	0.0	43	8	20.51	130.0	3.433721	0.197542
3	0.0	51	10	20.51	132.0	3.351107	0.135033
4	0.0	51	16	20.50	135.0	3.238121	0.107690

	target
0	0
1	0
2	0
3	0
4	0

```
[9]: # Splitting the dataset into the Training set and Test set
```

```
X_train, X_test, y_train, y_test = train_test_split(df[features], df[target],  
↳ test_size = 0.2, random_state = 0)
```

```
[10]: # No. of instance with 1 in target variable for training set
```

```
display(y_train.value_counts())
```

```
# No. of instance with 1 in target variable for test set
```

```
display(y_test.value_counts())
```

```
target  
0    2218  
1    1686  
Name: count, dtype: int64  
  
target  
0    536  
1    441  
Name: count, dtype: int64
```

```
[11]: # display the data after splitting
```

```
# Training set
```

```
display(pd.DataFrame(X_train).head())
```

```
display(pd.DataFrame(y_train).head())
```

```
# Test set
```

```
display(pd.DataFrame(X_test).head())
display(pd.DataFrame(y_test).head())
```

	precip	max_temp	min_temp	gwl	daily_discharge_cfs	sm_pca	\
641	0.00	57	11	21.00	145.0	0.494067	
333	0.11	35	19	20.84	93.7	-0.995163	
33	0.08	40	25	20.44	120.0	-0.179378	
3909	0.00	41	8	21.71	96.9	-0.002604	
4193	0.00	62	32	22.13	150.0	0.943268	

	st_pca
641	-1.976446
333	0.067906
33	1.826560
3909	1.751204
4193	-1.191578

	target
641	1
333	0
33	0
3909	1
4193	1

	precip	max_temp	min_temp	gwl	daily_discharge_cfs	sm_pca	\
2857	0.01	47	22	21.16	170.0	2.224886	
2373	0.00	75	30	21.73	100.0	0.893778	
1065	0.00	48	17	21.00	78.3	0.303675	
2287	0.00	97	60	21.31	112.0	-0.647494	
1675	0.00	80	42	21.45	163.0	-0.127969	

	st_pca
2857	2.490720
2373	-1.718663
1065	0.066434
2287	-0.541322
1675	-1.816134

	target
2857	0
2373	0
1065	1
2287	0
1675	0

```
[13]: # Scaling the features
      # Feature scaling is performed to ensure that the model is not biased towards
      # the features with higher magnitude
```

```
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
```

[14]: *# Displaying the scaled training set*

```
display(pd.DataFrame(X_train).head())
```

	0	1	2	3	4	5	6
0	-0.236413	-0.703677	-1.472493	-0.789317	-0.043247	0.398965	-1.188105
1	0.574686	-1.801402	-1.034528	-1.157828	-0.211894	-0.693305	0.026048
2	0.353477	-1.551919	-0.706055	-2.079106	-0.125433	-0.094971	1.070523
3	-0.236413	-1.502023	-1.636730	0.845953	-0.201374	0.034684	1.025768
4	-0.236413	-0.454194	-0.322836	1.813295	-0.026809	0.728431	-0.721967

[21]: *# Model building*

```
classifier_linear = SVC(kernel='linear')
#classifier_poly = SVC(kernel='poly')
#classifier_rbf = SVC(kernel='rbf')

# Fitting the model
classifier_linear.fit(X_train, y_train)
#classifier_poly.fit(X_train, y_train)
#classifier_rbf.fit(X_train, y_train)

# Predicting the Test set results
pred_linear = classifier_linear.predict(sc.transform(X_test))
#pred_poly = classifier_poly.predict(sc.transform(X_test))
#pred_rbf = classifier_rbf.predict(sc.transform(X_test))
```

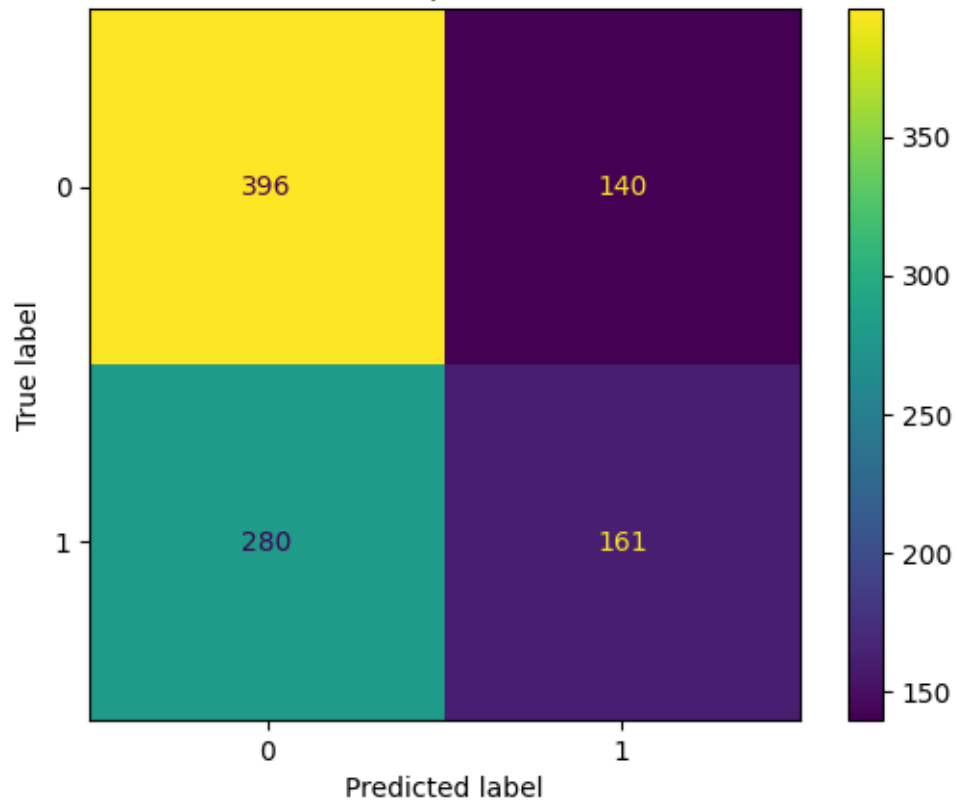
[22]: *# Generating the confusion matrix for the linear kernel*

```
cm = confusion_matrix(y_test, pred_linear)

disp = ConfusionMatrixDisplay(confusion_matrix=cm,
    ↪display_labels=classifier_linear.classes_)
disp.plot()
disp.ax_.set_title('Confusion Matrix for Simple SVM with Linear Kernel')
```

[22]: Text(0.5, 1.0, 'Confusion Matrix for Simple SVM with Linear Kernel')

Confusion Matrix for Simple SVM with Linear Kernel



[23]: *# Calculating the accuracy, f1 score and recall for the linear kernel*

```
accuracy_linear = accuracy_score(y_test, pred_linear)
f1_linear = f1_score(y_test, pred_linear)
recall_linear = recall_score(y_test, pred_linear)
```

Adding the scores to a dataframe

```
scores = pd.DataFrame({'Kernel': ['Linear']})
scores['Accuracy'] = [accuracy_linear]
scores['F1 Score'] = [f1_linear]
scores['Recall'] = [recall_linear]
```

Displaying the scores

```
display(scores)
```

	Kernel	Accuracy	F1 Score	Recall
0	Linear	0.570113	0.433962	0.365079

0.0.1 Model Optimization

Optimizing the model using GridSearchCV for Accuracy Metric Optimizing a Support Vector Machine (SVM) model using `GridSearchCV` for the accuracy metric involves systematically exploring a range of **hyperparameters** to find the combination that yields the **highest accuracy on your dataset**. Here's a step-by-step breakdown of what this entails:

- 1) Define the Hyperparameter Grid:

For SVMs, common hyperparameters to tune include the C parameter (regularization), gamma (kernel coefficient for 'rbf', 'poly', and 'sigmoid' kernels), and the kernel type itself (linear, poly, rbf, sigmoid). You need to create a grid of values for these hyperparameters to test. For example:

C: [0.1, 1, 10, 100]

gamma: [0.001, 0.01, 0.1, 1]

kernel: ['linear', 'rbf']

- 2) Use the `GridSearchCV` class from scikit-learn to perform an exhaustive search over the specified hyperparameter grid. You also specify the performance metric to optimize, which in this case is accuracy.
- 3) Fit the Model: Train the model using the fit method of `GridSearchCV`, which will evaluate all possible combinations of hyperparameters using cross-validation.
- 4) Select the Best Model: `GridSearchCV` will determine the best combination of hyperparameters based on the accuracy metric. The best model is the one that achieved the highest average accuracy across the cross-validation folds
- 5) Evaluate the Best Model: After identifying the best parameters, you can use this model to make predictions on your test data and evaluate its performance.

All in all, optimizing an SVM model using `GridSearchCV` for accuracy involves defining a range of hyperparameters to explore, setting up the grid search with accuracy as the scoring metric, fitting the model, and selecting the best-performing hyperparameter combination based on the accuracy achieved through cross-validation.

Optimizing the model using GridSearchCV for Accuracy Metric

```
[24]: # Implementing SVM with GridSearchCV for hyperparameter tuning of linear, rbf
      ↪and poly kernels

params_grid = [{ 'C': [0.1, 0.5, 1, 5, 10], 'kernel': ['linear'] },
               { 'C': [0.1, 0.5, 1, 5, 10], 'kernel': ['rbf'] },
               ↪{ 'C': [0.1, 0.5, 1, 5, 10], 'kernel': ['poly'], 'degree': [2, 3,
               ↪4] }]

classifier = SVC()
grid_search = GridSearchCV(estimator = classifier,
                           param_grid = params_grid,
                           scoring = 'accuracy', # accuracy is used as scoring
                           ↪metric)
```

```
        cv = 10,  
        n_jobs = -1)  
  
grid_search.fit(X_train, y_train)  
  
# Displaying the best parameters  
best_accuracy = grid_search.best_score_  
best_parameters = grid_search.best_params_  
print('Best Accuracy:', best_accuracy)  
print('Best Parameters:', best_parameters)
```

Best Accuracy: 0.8301678798609744

Best Parameters: {'C': 10, 'kernel': 'rbf'}